Laboratorio de Software
Práctica nº 4

Temas

- Arreglos de primitivos y objetos.
- Clases de la API para manejar colecciones de objetos
- Genéricos
- Expresiones Lambda

1.- Se quiere mantener un conjunto de alumnos ordenados por número de legajo, correspondientes a una materia en una Facultad. Defina una clase **`Alumno`** en el paquete **`practica4`**, que contenga información del alumno, entre otras cosas el legajo, apellido y nombre y DNI.

a) Defina una clase llamada **`Materia`** en el mismo paquete que mantenga información sobre la materia y la nómina de alumnos. Use colecciones genéricas.
b) Defina en la clase **`Materia`** 2 métodos: **`agregaAlumno(Alumno a)`** que agregue a alumnos a la nómina de alumnos existentes y **`getAlumnos()`** que retorne el conjunto de alumnos inscriptos en la materia.
c) Pruebe las clases definidas creando agregando al menos 10 alumnos en una **`Materia`**. Recorra la colección e imprima sus elementos.
d) Su colección está ordenada, ¿qué requisito tuvo que cumplir para poder mantenerla ordenada?
e) Si quisiera que el criterio de orden se mantenga sobre el apellido y nombre ¿cómo lo haría?
f) Analice una solución que permita mantener una nómina no sólo de alumnos sino también de empleados, clientes, etc. ¿Cómo definiría la colección?

2.- Considere la siguiente clase:

```
public class Veterinaria<E> {
    private E animal;

    public void setAnimal(E x) {
        animal = x;
    }
    public E getAnimal() {
        return animal;
    }
}

public class Animal{
}

public class Gato extends Animal {
}

public class Perro extends Animal {
}
```

a) Indicar cual es el resultado de las siguientes operaciones:

```
i)    Veterinaria <Animal> vet = new Veterinaria <Gato>();
ii)   Veterinaria <Gato> vet = new Veterinaria <Animal>();
iii)  Veterinaria <?> vet = new Veterinaria<Gato>();
      vet.setAnimal(new Gato());
iv)   Veterinaria vet = new Veterinaria ();
      vet.setAnimal(new Perro());
v)    Veterinaria vet = new Veterinaria <?>();
vi)   Veterinaria <? extends Animal> vet = new Veterinaria<Gato>();
```

3.- Implementación de **Expresiones Lambda en Java**

a) Defina una clase llamada **Facultad** que contenga en su interior una lista de alumnos, donde de estos últimos se guarda la siguiente información:

- nro de alumno
- nombres
- apellidos
- edad
- materia aprobada
- nota de aprobación

Sobre la clase **Facultad** implemente los métodos necesarios utilizando expresiones Lambda a fin de poder realizar las siguientes consultas:

1. Obtener el estudiante con mayor nota.
2. Imprimir dos estudiantes de la lista.
3. El que tomó el curso llamado "Laboratorio de Software".
4. Obtener los alumnos, cuyo nombre empiece con el carácter "P" y la longitud de su nombre sea menor o igual a 6.

Finalmente implemente una clase **TestFacultad** que permita probar las consultas anteriores.

b) Implemente el método **ordenarPorNota()** en la clase **Facultad** del ejercicio anterior. Dicho método ordena la lista de estudiantes por nota de aprobación de mayor a menor utilizando una clase interna que implementa la interface **java.util.Comparator**.

1. Modifique el código para reemplazar la clase interna por una expresión lambda.
2. Modifique la implementación para que utilice el método estático **Comparator.comparingInt()**. ¿Qué recibe como parámetro?
3. ¿Es posible utilizar una referencia a método? Utilícela en caso de ser posible.